

Cascaded Vector Similarity Decomposition for Vector-Augmented Analytical Queries

팀명: 속도는벡터
팀원: 박세은(팀원 1, 팀장), 강재현(팀원 2), 조현빈(팀원 3), 이동욱(팀원 4)
지도교수: 박광현
상태: 초안 — 3/31 팀 논의 후 확정 → 4/1 작성 마무리 → 4/2 제출

팀 논의 사항 (제출 전 확정 필요)

번호	항목	요약	추천
A-1	Cascade 메커니즘	D_loose를 선택도 ~30~40%에 대응하도록 설정 + Stage 2는 추정 불필요	메커니즘 확인 필요
A-2	Planning Time 미미 가능성	PG는 보통 1% 미만. 대안 경로(pre-filtering) 동등 서술	현행 유지
A-3	pg_hint_plan 반론	"정답을 아는 상태"에서만 가능 → Cascade가 더 범용적	제안서 반영 완료
A-4	RQ2/RQ3 구분	RQ3를 "ECQO 보완/대체" 질문으로 차별화	3개 유지
B-1	데이터 규모	SF1(~1GB) + SIFT1M(~512MB) ≈ 1.5GB로 시작, 차이 미미 시 SF10 확대	SF1 먼저
B-2	Exqutor 빌드 실패	ECQO 비교 불가 → Baseline vs Cascade only로 축소	4/7까지 결정
B-3	확장 실험 우선순위	top-k 1순위, pgvector 0.8.0 2순위	top-k 먼저
B-4	HNSW + range query	pgvector에서 range query가 HNSW를 타는지 확인 필요	빌드 후 즉시 확인
B-5	CTE 인라인 검증	PG 12+에서 비재귀 CTE 자동 인라인 → MATERIALIZED 없이 분해 효과 사라질 수 있음. EXPLAIN으로 검증 필요	환경 구축 직후 확인

A. 내용/방향 결정

A-1. Cascade의 작동 메커니즘 — 왜 Stage 1의 추정 오차가 작아지는가?

제안서 Problem Description에 아래 논리를 서술했는데, 이게 맞는 건지 논의 필요:

- pgvector는 벡터 조건의 선택도를 **33.3%로 고정** 추정함
- Stage 1의 D_loose를 **실제 선택도가 ~30~40%가 되도록 설정**하면, 고정 추정값과 실제값이 근접해서 Q-error ≈ 10 이 됨
- Stage 2는 Materialized CTE 결과를 스캔하는 것이므로 **벡터 카디널리티 추정 자체가 불필요함** (일반 테이블 스캔)
- 결과적으로 "추정 오차가 작은 단계 + 추정이 불필요한 단계"로 분리하는 것이 Cascade의 핵심

이 전제가 맞으려면 "D_loose를 어떻게 정하느냐"가 중요한데, 실제 데이터에서 선택도 33.3%에 대응하는 D 값을 사전에 알 수 있는지, 아니면 실험적으로 sweep해서 찾아야 하는지 확인 필요. 또한 pgvector가 Materialized CTE의 결과에 대해서도 벡터 추정을 시도하는지 여부도 실험 전에 확인해야 함. → **결정 필요**

A-2. Planning Time이 미미할 가능성이 높음

설계안에서도 인정한 부분: "PostgreSQL의 Planning Time은 일반적으로 Execution Time의 1% 미만." 가설 H1($\geq 30\%$)이 기각될 가능성이 꽤 높음. 현재 제안서는 "일단 측정해보자"를 유지하면서, 기각되더라도 "pre-filtering을 통한 실행 시간 단축" 방향으로 전환하는 대안 경로를 동등하게 서술했음. 어느 쪽이든 1단계 프로파일링 자체가 독립적 기여가 되므로, 이 프레이밍으로 가는 게 맞는지 확인. → **현행 유지 추천**, 1단계 결과에 따라 판가름

A-3. "pg_hint_plan으로 최적 계획을 강제하면 되지 않나?"에 대한 답변

Cascade의 가치에 대해 나올 수 있는 가장 날카로운 반론. 제안서에는 이렇게 반영해둬: "pg_hint_plan은 정답을 이미 아는 상태에서만 가능하고, 실무에서는 쿼리마다 최적 계획을 사전에 알 수 없으므로 구조적으로 추정 오차를 줄이는 접근이 더 범용적이다. 본 연구에서 pg_hint_plan은 oracle plan(이론적 최적)을 확인하는 도구로만 사용한다." 이 답변이 충분한지, 추가 보완이 필요한지 확인. → **반영 완료, 추가 의견 있으면 논의**

A-4. RQ2와 RQ3의 구분이 약한 문제

RQ2("분해하면 성능이 개선되는가?")와 RQ3("총 비용이 ECQO 대비 어떠한가?")가 사실상 비슷한 질문이라는 지적이 가능함. 현재는 RQ3를 "ECQO와 보완적인가, 대체적인가?"로 차별화하여 독자적 질문으로 만들어둬. RQ 3개 = 실험 3단계 1:1 대응 구조가 깔끔하므로 이대로 가는 게 나을 듯. → **3개 유지 추천, 다른 의견 있으면 논의**

B. 실행 가능성 결정

B-1. 데이터 규모 — SF1(~1GB)에서 유의미한 차이를 관찰할 수 있는가?

TPC-H SF1(~1GB) + SIFT1M(128d, ~512MB) ≈ 1.5GB 합산. 이 규모면 전체 데이터가 PostgreSQL shared_buffers에 들어가서 I/O 차이가 사라지고, Planning Time은 sub-millisecond 수준일 가능성이 높음. 유의미한 Planning Time 차이를 관찰하려면 조인 테이블을 늘려 플래닝 공간을 키우거나, SF10(~10GB, 서버 필요)으로 확대해야 할 수 있음. → SF1로 시작해서 차이 미미하면 조인 복잡도 높이기 또는 SF10 확대 추천. 서버 사용 가능 여부도 같이 확인 필요

B-2. Exqutor 패치 빌드가 실패하면?

제안서의 2단계 Phase 2a 4전략 중 2개(ECQO only, ECQO + Cascade), 3단계 전체가 ECQO 비교에 의존함. 빌드 실패 시 Baseline vs Cascade only 비교만 남아서 연구 범위가 상당히 축소됨. 대학원생에게 빌드된 바이너리나 Docker 환경을 직접 요청하는 게 현실적. → 4/7까지 대학원생 컨택 결과에 따라 결정. 누가 연락할지도 정해야 함

B-3. 확장 실험 우선순위

3단계 확장 실험 후보: (1) top-k 전환 — 실무에서 거의 전부 top-k이므로 관련성 최고, `ORDER BY dist LIMIT k`로 쿼리만 바꾸면 되어 구현도 간단. (2) pgvector 0.8.0 비교 — iterative scan 등 자체 개선이 Cascade를 대체할 수 있는지 시의성 높음. (3) 고차원(768d) — 논문 한계 검증이지만 우선순위 낮음. → top-k 1순위, pgvector 0.8.0 2순위 추천

B-4. pgvector에서 range query가 HNSW 인덱스를 타는가?

Cascade Stage 1의 `WHERE embedding <-> query < D_loose`가 HNSW 인덱스를 활용하는지 여부가 핵심 전제. pgvector 0.7.x 기준으로 range query는 HNSW를 안 타고 Sequential Scan으로 갈 가능성이 있음. Exqutor 패치가 이를 해결하는 핵심 기능인데, 패치 없이도 인덱스가 타는지 빌드 후 `EXPLAIN`으로 즉시 확인해야 함. 만약 안 타면 Stage 1의 속도 이점이 사라지므로, IVFFlat 인덱스 또는 다른 전략이 필요할 수 있음. → 환경 구축 직후 최우선 확인

[Contents]

1. Background
 - 1.1. Vector-Augmented Analytical Queries (VAQ)
 - 1.2. Exqutor: Cardinality Estimation for VAQ
 - 1.3. Cascaded Filtering in Database Systems
2. Problem Description
3. Research Plan
4. Expected Contribution
5. Team Member
6. Future Schedule
7. References

1. Background

1.1. Vector-Augmented Analytical Queries (VAQ)

벡터 증강 분석 쿼리(Vector-Augmented Analytical Query, VAQ)는 전통적인 관계형 분석 쿼리에 벡터 유사도 검색을 결합한 새로운 유형의 쿼리이다. RAG(Retrieval-Augmented Generation) 파이프라인, 추천 시스템, 멀티 모달 검색 등 다양한 응용에서 "벡터 유사도 조건 + 관계형 필터 + 집계/조인"이 결합된 쿼리가 광범위하게 활용되고 있다.

예를 들어, 전자상거래에서 "쿼리 벡터와 유사한 상품 중 특정 가격대와 카테고리에 해당하는 것의 주문 건수와 평균 금액을 집계"하는 쿼리는 다음과 같다:

```
SELECT p.category, COUNT(*), AVG(o.amount)
FROM products p JOIN orders o ON p.id = o.product_id
WHERE p.embedding <-> :query < :D
      AND p.category = 'electronics'
      AND p.price BETWEEN 100 AND 500
GROUP BY p.category;
```

이러한 VAQ에서 핵심 문제는 **벡터 유사도 조건의 카디널리티(결과 건수)**를 쿼리 옵티마이저가 **정확히 추정하지 못한다**는 것이다. pgvector는 벡터 조건의 선택도를 33.3%로 고정 추정하고, VBASE는 50%, DuckDB는 100%로 추정한다 [1]. 실제 선택도는 0.001%에서 90%까지 극도로 다양하므로, 옵티마이저가 잘못된 실행 계획(예: 불필요한 Sequential Scan, 잘못된 조인 순서)을 선택하여 성능이 크게 저하된다. Leis et al. [5]이 제시한 Q-error($\max(\text{estimated}/\text{actual}, \text{actual}/\text{estimated})$) 기준으로 보면, 이러한 고정 추정은 선택도가 낮을수록 수백~수천 배의 추정 오차를 발생시킨다.

1.2. Exqutor: Cardinality Estimation for VAQ

Exqutor [1]는 이 카디널리티 오추정 문제를 두 가지 방법으로 해결한다.

ECQO (Exact Cardinality Query Optimization): HNSW 인덱스가 존재할 때, range query를 인덱스 위에서 실행하여 1~2ms 안에 정확한 카디널리티를 획득한다. 이 정확한 카디널리티를 옵티마이저에 전달하면 올바른 실행 계획을 선택하게 된다.

Adaptive Sampling: 인덱스가 없을 때, 모멘텀 기반 동적 샘플링으로 카디널리티를 근사한다. 샘플 크기를 적응적으로 조절하여 오버헤드를 최소화하면서도 충분한 정확도를 확보한다.

그 결과 pgvector에서 최대 1,000배, VBASE에서 10,000배, DuckDB에서 1.5~37배의 성능 향상을 달성했다. 그러나 Exqutor의 실험에는 다음과 같은 고정 조건들이 존재한다:

변수	논문이 사용한 값	현실과의 괴리
쿼리 유형	range query (<code>WHERE dist < D</code>)	실무 RAG는 거의 전부 top-k
벡터 조건	단일 벡터 유사도 조건 1개	벡터 조건의 구조적 변환 미탐구
벡터 차원	96~128d (DEEP 96d, SIFT 128d)	OpenAI 1536d, Cohere 4096d
거리 함수	L2(유클리드)만	코사인 유사도가 더 흔함
병목 분석	실행 시간 위주	플래닝 시간 vs 실행 시간 분리 분석 없음

특히 Exqutor는 VAQ의 전체 지연(latency) 중 플래닝 단계와 실행 단계가 각각 어느 정도를 차지하는지 분리 분석하지 않았다. 이 분리 분석은 성능 최적화의 방향을 결정하는 선행 조건이다.

1.3. Cascaded Filtering in Database Systems

데이터베이스 분야에서 **Cascaded Filtering** 또는 **Coarse-to-Fine Search**는 오래된 패러다임이다. ANN 검색에서 coarse quantizer → fine re-ranking 같은 다단계 파이프라인이 표준이며 [2], 쿼리 실행 중에 카디널리티를 점진적으로 개선하는 Progressive Optimization [3], 선택도에 따라 검색 범위를 동적으로 조정하는 Filtered-DiskANN [4] 등의 연구가 있다.

그러나 벡터 유사도 조건 자체를 쿼리 수준에서 두 단계로 분해하여, 옵티마이저의 추정 오차가 작아지는 구조를 만들거나 중간 결과 캐싱을 통해 실행을 가속하는 접근은 기존 연구에서 다뤄지지 않았다.

2. Problem Description

본 연구가 다루는 문제는 다음과 같다.

VAQ에서 벡터 유사도 조건의 카디널리티 불확실성은 쿼리 성능에 두 가지 경로로 영향을 미친다. 첫째, 옵티마이저가 잘못된 실행 계획을 선택한다(플래닝 품질 저하). 둘째, 잘못된 계획에 따라 불필요하게 많은 데이터를 처리하게 된다(실행 효율 저하). Exqutor는 카디널리티를 정확히 추정하여 첫 번째 문제를 해결했다. 본 연구는 벡터 조건의 구조 자체를 변환하여, 추정이 부정확하더라도 그 영향이 최소화되는 쿼리 구조를 탐구한다.

단일 벡터 유사도 조건 `WHERE embedding <-> query < D` 에서:

- 옵티마이저는 D 값에 따른 결과 건수를 알 수 없어 실행 계획 수립이 어렵다
- D가 작으면(선택도↓) 결과가 매우 적는데 전체 테이블을 스캔할 수 있고, D가 크면(선택도↑) 결과가 많은데 인덱스만 타려 할 수 있다
- 이 불확실성은 관계형 필터와 조인이 추가될수록 플래닝 공간을 확장시킨다

만약 이 벡터 조건을 **느슨한 임계값(Stage 1)**과 **엄격한 임계값(Stage 2)** 두 단계로 분해한다면:

- Stage 1의 D_loose를 실제 선택도가 pgvector 기본 추정치(33.3%)에 근접하도록 설정하면, 고정 추정값과 실제값의 괴리가 최소화된다. D_loose는 1단계 선택도 sweep 결과에서 선택도 ~33%에 대응하는 거리 임계값을 역산하거나, 소규모 사전 샘플링으로 근사하여 결정한다. 사전 샘플링의 구체적 방법과 오버헤드는 1단계 실험에서 확정한다(상세는 3절 RQ1 참조). 다만 이 효과는 벡터 조건 단독일 때 가장 명확하며, 관계형 필터가 추가되면 전체 선택도 추정에 여전히 오차가 발생할 수 있으므로 복합 필터 조건에서의 효과는 별도 검증이 필요하다.
- Stage 2는 Materialized CTE 결과를 스캔하므로 벡터 카디널리티 추정의 오차 영향이 구조적으로 최소화된다. Stage 2의 추정 대상이 전체 테이블이 아닌 실체화된 소규모 결과 집합이므로, 추정 오차의 절대적 영향이 크게 줄어든다.
- 결과적으로 "추정 오차가 작은 단계 + 추정 오차의 영향이 최소화된 단계"로 분리되어, 옵티마이저가 올바른 계획을 선택할 가능성이 높아진다
- 부가적으로, Stage 1의 결과를 캐싱하면 Stage 2는 해당 결과 내에서만 검색하므로 실행 효율도 개선될 수 있다

이것이 본 연구의 핵심 아이디어인 **Cascaded Vector Similarity Decomposition**이다. 이 아이디어는 본 팀 강재현의 관찰에서 출발하였으며, 데이터베이스의 Cascaded Filtering 패러다임과 ANN 검색의 Coarse-to-Fine 파이프라인의 원리를 벡터 유사도 조건의 쿼리 수준 분해에 적용한 것이다. 아래 그림은 이 분해 구조의 전체 흐름을 나타낸다.

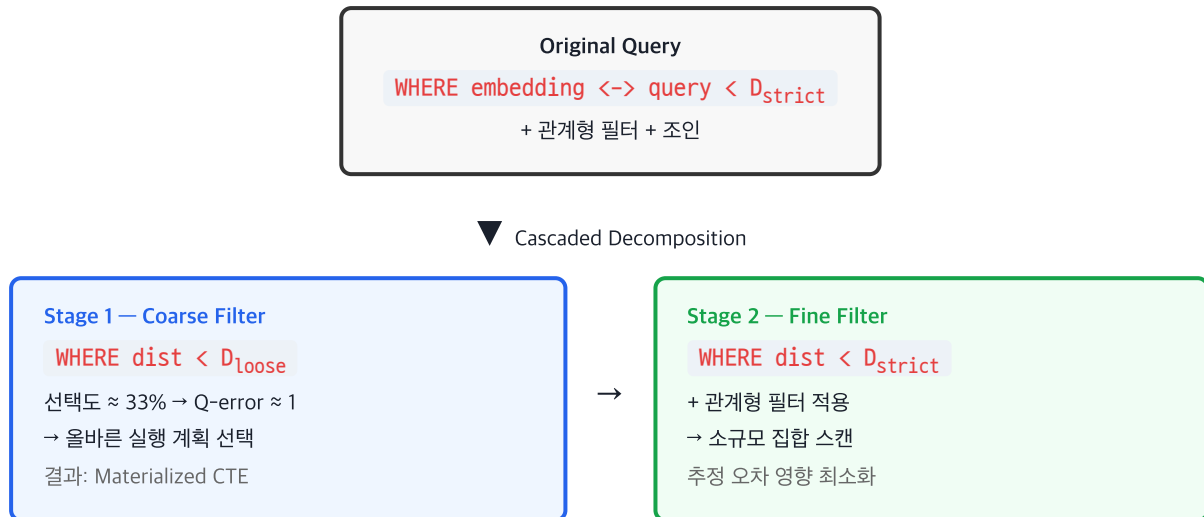


Figure 1. Cascaded Vector Similarity Decomposition의 2단계 분해 구조. Stage 1은 느슨한 임계값으로 후보 집합을 생성하고, Stage 2는 실제화된 결과 내에서 정밀 필터링을 수행한다.

3. Research Plan

본 연구는 세 개의 연구 질문(RQ)에 대응하는 3단계 실험으로 구성된다. 실험 데이터셋은 TPC-H SF1에 SIFT1M(128d) 벡터를 매핑한 VAQ 벤치마크를 기본으로 사용하며, PostgreSQL 16 + pgvector 0.7.x 환경에서 수행한다.

RQ1: VAQ에서 플래닝 시간과 실행 시간의 병목 구조는 어떠한가?

1단계 — 병목 프로파일링

Cascade 분해의 효과가 어느 경로(계획 품질 개선 vs 실행 가속)에서 발현되는지를 결정하기 위해, 먼저 VAQ의 성능 병목 구조를 정량적으로 분석한다. PostgreSQL의 `EXPLAIN (ANALYZE, TIMING, BUFFERS, FORMAT JSON)` 으로 Planning Time과 Execution Time을 분리 측정한다.

- **선택도 sweep**: 거리 임계값 D 를 조절하여 벡터 선택도를 0.01%~90%로 변화시키며, 각 지점에서 Planning Time / Execution Time 비율, Q-error [5](#) 선택된 실행 계획을 기록한다. 이 sweep 결과는 동시에 2단계에서 `D_loose` 설정의 근거 데이터로 활용된다.
- **필터 복잡도별 변화**: 관계형 필터를 단계적으로 추가(0개 → 1개 → 2개 → 3개+조인)하면서 플래닝 시간 및 실행 시간의 증가율을 측정한다.

이 단계의 결과에 따라 2단계 실험의 초점이 결정된다:

- **플래닝이 유의미한 병목인 경우** (Planning Time $\geq$ 전체 latency의 20%): Cascade 분해의 효과를 "계획 품질 개선"과 "실행 가속" 양쪽에서 검증
- **플래닝이 미미한 경우** (Planning Time < 전체 latency의 10%): Cascade의 효과를 "Stage 10 pre-filter 역할을 하여 Stage 2의 실행 시간을 단축하는 구조적 이점"에 집중. end-to-end latency(Planning Time + Execution Time 합계) 비교가 핵심 지표가 된다.

어느 경우든 1단계 자체가 "VAQ에서 선택도에 따른 플래닝/실행 비중의 정량적 프로파일"이라는 독립적 기여를 갖는다.

RQ2: 벡터 유사도 조건을 두 단계로 분해하면 쿼리 성능이 개선되는가?

2단계 — Cascaded Vector Similarity Decomposition

1단계와 동일한 TPC-H SF1 + SIFT1M 데이터셋 위에서, 제안하는 분해 방식을 적용한다:

```
-- Stage 1: 느슨한 임계값으로 후보 집합 생성
WITH MATERIALIZED coarse_set AS (
  SELECT *, embedding <-> :query AS dist
  FROM items WHERE embedding <-> :query < D_loose
)
-- Stage 2: 후보 집합 내에서 엄격한 조건 적용
SELECT * FROM coarse_set
WHERE dist < D_strict
  AND category = 'electronics'
  AND price BETWEEN 100 AND 500;
```

PostgreSQL 12+에서 비재귀 CTE는 기본적으로 자동 인라인된다(optimization fence 없음). **MATERIALIZED** 키워드를 명시하면 인라인을 방지하고 중간 결과의 실체화를 강제할 수 있다. 실체화 자체의 오버헤드(임시 테이블 기록)가 존재하므로, 이것이 성능 이득을 상쇄하지 않는지가 핵심 검증 대상이다.

실험은 조합 폭발을 방지하기 위해 Phase 2a → 2b의 2-phase 설계로 진행한다:

- **Phase 2a** (12~16개 조합): 1단계 sweep에서 도출한 분해 임계값 4가지(예: 선택도 5%/15%/33%/60%에 대응하는 D_loose) × 비교 전략 4가지(Baseline / ECQO only / Cascade only / ECQO + Cascade). 필터 없이 고정, 캐싱은 Materialized CTE만 사용.
- **Phase 2b** (24~36개 조합): Phase 2a에서 유의미한 조합(end-to-end latency 10% 이상 차이)만 선택 → 필터 4단계 × 캐싱 전략 3가지(CTE / Materialized CTE / Temp Table + Index)로 확장.

측정 프로토콜: 각 조합 최소 5회 실행, 첫 1회 워밍업 제외, 나머지 4회 중 최대값을 아웃라이어로 제거하고 3회 평균을 사용한다. 환경 통제를 위해 **pg_stat_statements** 리셋 후 단독 세션에서 실행하며, shared_buffers 워밍업을 보장한다. pg_hint_plan은 oracle plan(이론적 최적)을 확인하는 도구로만 사용하며, Cascade는 pg_hint_plan 없이도 올바른 계획을 유도하는 구조적 접근으로서의 가치를 검증한다.

RQ3: Cascade와 Exqutor ECQO는 보완적인가, 대체적인가?

3단계 — ECQO 상호작용 분석 및 확장 실험

end-to-end latency(Planning Time + Execution Time)를 기준으로 네 전략을 비교한다:

비교 전략	설명
Baseline	pgvector 기본 (33.3% 고정 추정)
ECQO only	Exqutor의 정확한 카디널리티 추정
Cascade only	벡터 조건 분해 (ECQO 없이)
ECQO + Cascade	정확한 카디널리티 + 벡터 조건 분해

ECQO는 "정확한 카디널리티를 줘서 올바른 계획을 선택하게 하는" 접근이고, Cascade는 "카디널리티를 몰라도 영향이 작은 구조를 만드는" 접근이다. 만약 ECQO + Cascade가 ECQO only보다 추가 향상을 보이면 두 접근은 보완적이며, 차이가 없으면 ECQO가 Cascade를 대체한다.

여력에 따라 확장 실험을 1~2개 추가 수행한다:

- **top-k 전환** (1순위): range query에서 확인한 효과가 `ORDER BY dist LIMIT k` 에서도 유효한지
- **pgvector 0.8.0 비교** (2순위): iterative scan 등 자체 개선으로 Cascade의 필요성이 감소하는지

4. Expected Contribution

본 연구의 기여는 다음과 같다.

첫째(RQ1), **VAQ의 성능 병목을 Planning Time과 Execution Time으로 분리하여 정량적으로 분석한다.** Exqutor를 포함한 기존 연구는 end-to-end latency만 비교했으며, 플래닝 단계(Planning Time)의 오버헤드를 독립적으로 분석하지 않았다. 본 연구는 선택도와 필터 복잡도에 따른 병목 구조를 정량화한다.

둘째(RQ2), **벡터 유사도 조건을 쿼리 수준에서 구조적으로 분해하는 새로운 최적화 접근(Cascaded Vector Similarity Decomposition)**을 제안하고 실험적으로 검증한다. 이는 카디널리티 추정의 정확도를 높이는 것이 아니라, 추정이 부정확하더라도 영향이 최소화되는 쿼리 구조를 만드는 접근이다.

셋째(RQ3), **Exqutor ECQO와 Cascade 분해의 상호보완성 또는 대체성**을 실험적으로 판별하여, 조건별 최적 전략을 제시하는 실무 가이드라인을 제공한다.

5. Team Member

구분	이름	비고
팀원 1	박세은	팀장
팀원 2	강재현	핵심 아이디어 제안
팀원 3	조현빈	
팀원 4	이동욱	

- 지도교수: 박광현

현재 환경 구축 단계로 역할은 유동적으로 배분하며, 전원이 모든 단계에 참여한다.

6. Future Schedule

주차	기간	내용
3	3/31	연구 방향 확정 (팀 논의)
3	4/1	제안서/계획서 작성 마무리
3	4/2	연구제안서 + 수행계획서 제출
3~5	4/1~4/14	환경 구축 (PostgreSQL 16 + Exqutor 패치 + pgvector + TPC-H VAQ 데이터)
3~5	4/14~4/28	1단계: 병목 프로파일링 (Planning vs Execution 비율 곡선, Q-error 곡선)
5~7	4/28~5/15	2단계: Cascaded Decomposition 핵심 실험 (분해 임계값 최적점, ECQO 비교)
7~9	5/15~5/25	3단계: ECQO 상호작용 + 확장 실험 (top-k 전환, pgvector 0.8.0 비교 등)
9~10	5월 말	중간발표 + 중간보고서
10~15	6월	결과 정리 + 최종보고서 + 최종발표 + 전시회

7. References

- [1] H. Kim, C. Lim, H. An, R. Sen, and K. Park, "Exqutor: Extended Query Optimizer for Vector-Augmented Analytical Queries," *arXiv preprint arXiv:2512.09695v2*, 2025.
- [2] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2021.
- [3] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdžić, "Robust Query Processing through Progressive Optimization," in *Proc. ACM SIGMOD*, 2004.
- [4] S. Gollapudi, N. Kini, V. Sivashankar, R. Krishnaswamy, H. Simhadri, et al., "Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters," in *Proc. WWW*, 2023.
- [5] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How Good Are Query Optimizers, Really?" *Proc. VLDB Endowment*, vol. 9, no. 3, pp. 204-215, 2015.